

AI Assignment 6: Dynamic Wumpus Logic Agent

Submitted by: Nehza Nazir (23F-0822) | NUCES Chiniot-Faisalabad

1. Mandatory Links

GitHub Repo: <https://github.com/Nehza-Nazir/Wumpus-Logic-Agent>

Vercel / Live URL: <https://wumpus-logic-agent-weld.vercel.app/>

LinkedIn Post: [\[Paste your LinkedIn Post URL here\]](#)

2. Project Objective & Specifications

The objective of this project is to develop a Web-based Dynamic Pathfinding Agent that acts as a strictly Knowledge-Based Agent. The agent navigates a Wumpus World grid by receiving dynamic percepts (Breeze, Stench, Glitter) and utilizes Propositional Logic coupled with Resolution Refutation to deductively prove whether adjacent unvisited cells are safe to explore.

Key specifications implemented:

- Dynamic Grid Configuration: Users can define Rows, Columns, and number of Pits at runtime.
- Hidden Hazards: Pits and the Wumpus are placed randomly. The agent does NOT know their locations initially.
- Strict Logic Execution: The agent only moves to cells that have been mathematically proven safe via the Resolution engine.

3. Propositional Logic & CNF Conversion

The core of the Knowledge Base (KB) relies on maintaining rules in Conjunctive Normal Form (CNF). When the agent receives a percept (e.g., a Breeze) at a cell (r,c), it signifies that at least one adjacent cell (a1, a2, etc.) contains a Pit. Conversely, if an adjacent cell contains a Pit, the current cell MUST have a Breeze. This is represented as a biconditional rule:

$$\text{Breeze}(r,c) \iff \text{Pit}(a1) \vee \text{Pit}(a2) \vee \dots \vee \text{Pit}(an)$$

To enable programmatic Resolution, this biconditional is automatically translated into the following CNF clauses:

- $\neg \text{Breeze}(r,c) \vee \text{Pit}(a1) \vee \text{Pit}(a2) \dots$ (If there is a breeze, there is a pit in an adjacent cell)
- $\neg \text{Pit}(a1) \vee \text{Breeze}(r,c)$ (If there is a pit at a1, the current cell will have a breeze)
- $\neg \text{Pit}(a2) \vee \text{Breeze}(r,c)$ (If there is a pit at a2, the current cell will have a breeze)

AI Assignment 6: Dynamic Wumpus Logic Agent

Submitted by: Nehza Nazir (23F-0822) | NUCES Chiniot-Faisalabad

Furthermore, the percept itself is added as a unit clause. If a breeze is felt, $\{Breeze(r,c)\}$ is added. If no breeze is felt, $\{\sim Breeze(r,c)\}$ is added.

Optimization via Unit Propagation:

If no breeze is felt, the algorithm efficiently resolves $\{\sim Breeze(r,c)\}$ against $\{\sim Pit(ai) \vee Breeze(r,c)\}$ to immediately yield $\{\sim Pit(ai)\}$ for all adjacent cells, categorically proving them safe from pits in a single resolution step.

AI Assignment 6: Dynamic Wumpus Logic Agent

Submitted by: Nehza Nazir (23F-0822) | NUCES Chiniot-Faisalabad

4. Resolution Refutation Algorithm Implementation

Before making a move to any adjacent, unvisited cell, the agent MUST query its KB: "Is cell (r,c) safe?". A cell is safe only if we can prove both $\sim \text{Pit}(r,c)$ AND $\sim \text{Wumpus}(r,c)$. The python backend implements an automated Resolution Refutation Engine to prove these goals.

Proof by Contradiction Loop:

1. Negate the Goal: To prove $\sim \text{Pit}(r,c)$, the system assumes the opposite: $\text{Pit}(r,c)$ is True. It temporarily adds $\{\text{Pit}(r,c)\}$ to the KB.
2. Set-of-Support Strategy: Rather than performing an exhaustive $O(N^2)$ comparison of all clauses in the KB (which causes severe performance bottlenecks), the engine employs a 'Set-of-Support' strategy. It strictly resolves the negated goal (and any newly derived resolvents from it) against the existing KB clauses.
3. Resolution Rule: Two clauses containing complementary literals (e.g., L and $\sim L$) are merged, dropping the complementary pair.
4. Deriving the Empty Clause: If merging two clauses results in an empty set $\{\}$, a logical contradiction has been found. Because the KB facts are absolute, the contradiction proves that our assumption $\{\text{Pit}(r,c)\}$ was inherently false. Therefore, $\sim \text{Pit}(r,c)$ is mathematically proven.

If the algorithm reaches its maximum iteration limit without deriving the empty clause, the safety of the cell remains UNKNOWN. The agent is programmed to strictly avoid unknown cells to guarantee its survival.

5. System Architecture & UI Visualization

The project utilizes a modern decoupled architecture:

Backend (Python/Flask):

Contains the Knowledge Base class, the Resolution Engine, and the Agent loop. The logic is kept entirely stateless between API calls by rebuilding the KB from the agent's percept history during every step. This design makes it perfectly suited for Serverless deployment on Vercel.

Frontend (Vanilla HTML/CSS/JS):

A custom-built, premium 'Glassmorphism' dark-theme UI was developed to satisfy the visualization requirements without relying on bulky frameworks. It features:

- A dynamic grid mapping Safe cells (Green), Unknown cells (Gray), and Danger/Pits (Red).
- A Real-Time Metrics Dashboard tracking Agent Steps, Inference Steps, KB Clauses, and active percepts.
- A live Resolution Log that prints the mathematical proof outcomes generated by the backend during every decision cycle.